1. **Define File in C. Write the instructions in C for creating a file**

   In **C programming**, a **file** is a data storage resource used to **store information permanently**. It allows data to be read from or written to a **disk file**, rather than being lost after the program ends (like with variables in memory).

   C supports file handling using a set of standard library functions defined in the **<stdio.h>** header, and files are handled using a special **pointer of type FILE**.

   Instructions to Create a File in C:

```
#include <stdio.h>

int main() {
    FILE *fp;  // Declare a file pointer

    // Create (or open if it already exists) a file named
"example.txt" in write mode
    fp = fopen("example.txt", "w");

    // Check if the file was created successfully
    if (fp == NULL) {
        printf("Error creating the file!\n");
        return 1;  // Exit with error code
    }

    // Write content to the file
    fprintf(fp, "Hello, this is a file created in C.\n");

    // Close the file
    fclose(fp);

    printf("File created and data written successfully.\n");

    return 0;
}
```

2. **Define File Mode. Describe different types of file mode.**

   In **C programming**, **file mode** refers to the **way or purpose for which a file is opened**, such as reading, writing, or appending.
   The mode is specified as a **string argument** in the fopen() function to tell the system how to handle the file.

   Different Types of File Modes:

Mode	Description
"r"	Open for reading. The file must exist.
"w"	Open for writing. If the file exists, its contents are deleted. If not, a new file

	is created.
"a"	Open for appending. Data is written at the end. If the file does not exist, it is created.
"r+"	Open for both reading and writing. The file must exist.
"w+"	Open for both reading and writing. Deletes contents if file exists, or creates a new file.
"a+"	Open for both reading and appending. Reads from beginning, writes at the end. Creates the file if it doesn't exist.

3. **Describe the method for merging two files in c.**

Definition:
Merging two files in C means **combining the contents of two existing files into a third file**. This is commonly used when you want to consolidate data.

Steps/Method to Merge Two Files in C:

1. **Open the first file** in read mode.
2. **Open the second file** in read mode.
3. **Open a third file** in write mode (this will store the merged content).
4. **Read contents from both files** and write them into the third file.
5. **Close all files.**

***C* Program to Merge Two Files:**

```
#include <stdio.h>

int main() {
    FILE *fp1, *fp2, *fp3;
    char ch;

    // Open first file in read mode
    fp1 = fopen("file1.txt", "r");
    // Open second file in read mode
    fp2 = fopen("file2.txt", "r");
    // Open third file in write mode
    fp3 = fopen("merged.txt", "w");

    // Check if files opened successfully
    if (fp1 == NULL || fp2 == NULL || fp3 == NULL) {
        printf("Error opening files!\n");
        return 1;
    }

    // Copy content of first file to merged file
    while ((ch = fgetc(fp1)) != EOF) {
        fputc(ch, fp3);
    }
```

```
    // Copy content of second file to merged file
    while ((ch = fgetc(fp2)) != EOF) {
        fputc(ch, fp3);
    }

    // Close all files
    fclose(fp1);
    fclose(fp2);
    fclose(fp3);

    printf("Files merged successfully into 'merged.txt'.\n");

    return 0;
}
```

4. **Create a program to copy file fl.dat assuming fl.dat is available in your current working directory. Explain the output of this program.**

```
#include <stdio.h>

int main() {
    FILE *src, *dest;
    char ch;

    // Open source file (fl.dat) in read mode
    src = fopen("fl.dat", "r");

    // Check if the source file exists
    if (src == NULL) {
        printf("Error: Source file 'fl.dat' not found!\n");
        return 1;
    }

    // Open destination file in write mode
    dest = fopen("copy_fl.dat", "w");

    // Copy contents character by character
    while ((ch = fgetc(src)) != EOF) {
        fputc(ch, dest);
    }

    // Close both files
    fclose(src);
    fclose(dest);

    printf("File copied successfully to 'copy_fl.dat'.\n");

    return 0;
}
```

5. **Write the significance of EOF.**

In C, EOF stands for **End Of File**. It is a **special constant** defined in the <stdio.h> header, usually having the value -1. It is used to **detect the end of input or file data** during reading operations.

Significance of EOF:

1. **File Reading Termination**:
   EOF helps determine **when to stop reading** from a file using functions like fgetc(), fgets(), or fscanf().
2. **Input Stream Control**:
   In input operations, EOF can signal **the end of user input** (e.g., by pressing Ctrl+Z on Windows or Ctrl+D on Linux/macOS in console).
3. **Loop Conditions**:
   Commonly used in loops to process data until end of file:
   ```
   while ((ch = fgetc(fp)) != EOF) {
       // process character
   }
   ```
4. **Error Handling**:
   EOF can be used to detect if a file could **not be read properly** or if an attempt to read **beyond file content** is made.
5. **Portable Programming**:
   EOF provides a standard and platform-independent way to detect end-of-file situations.

Example:

```
#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    fp = fopen("data.txt", "r");

    if (fp == NULL) {
        printf("File not found!\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }

    fclose(fp);
    return 0;
}
```

9. **Explain the functionality of fseek(), ftell() , rewind, fprintf(), fscanf()**

1. **fseek(FILE *fp, long offset, int origin)**
   - **Purpose:** Moves the file pointer to a specific location in the file.
   - **Syntax Example:**

   fseek(fp, 0, SEEK_END); // Moves pointer to end of file
   - **Parameters:**
     - fp: File pointer
     - offset: Number of bytes to move
     - origin: Starting point (SEEK_SET, SEEK_CUR, or SEEK_END)

---

2. **ftell(FILE *fp)**
   - **Purpose:** Returns the **current position** of the file pointer in bytes.
   - **Syntax Example:**

   long pos = ftell(fp);
   - **Returns:** A long integer indicating the position in the file. Returns -1L on error.

---

3. **rewind(FILE *fp)**
   - **Purpose:** Sets the file pointer to the **beginning of the file**.
   - **Syntax Example:**

   rewind(fp);
   - **Note:** Equivalent to fseek(fp, 0, SEEK_SET); but also clears error and EOF flags.

---

4. **fprintf(FILE *fp, const char *format, ...)**
   - **Purpose:** Writes **formatted data** to a file (like printf() but to a file).
   - **Syntax Example:**

   fprintf(fp, "Name: %s, Age: %d\n", name, age);
   - **Use:** Useful for writing structured data to files.

---

5. **fscanf(FILE *fp, const char *format, ...)**
   - **Purpose:** Reads **formatted data** from a file (like scanf() but from a file).
   - **Syntax Example:**

   fscanf(fp, "%s %d", name, &age);
   - **Use:** Useful for reading structured data (like strings, ints) from a file.

10. Two files contain a sorted list of integers. Write a C program to produce a third file which holds a single sorted, merged list of these two lists. Use command line arguments to specify the file names.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *fp1, *fp2, *fp3;
    int num1, num2;
    int status1, status2;
```

```
    // Check if enough command line arguments are provided
    if (argc != 4) {
        printf("Usage: %s <input1.txt> <input2.txt> <output.txt>\n",
argv[0]);
        return 1;
    }

    // Open files
    fp1 = fopen(argv[1], "r");
    fp2 = fopen(argv[2], "r");
    fp3 = fopen(argv[3], "w");

    if (fp1 == NULL || fp2 == NULL || fp3 == NULL) {
        printf("Error opening files.\n");
        return 1;
    }

    // Read initial integers
    status1 = fscanf(fp1, "%d", &num1);
    status2 = fscanf(fp2, "%d", &num2);

    // Merge two sorted files
    while (status1 != EOF && status2 != EOF) {
        if (num1 <= num2) {
            fprintf(fp3, "%d ", num1);
            status1 = fscanf(fp1, "%d", &num1);
        } else {
            fprintf(fp3, "%d ", num2);
            status2 = fscanf(fp2, "%d", &num2);
        }
    }

    // Copy remaining elements from file1
    while (status1 != EOF) {
        fprintf(fp3, "%d ", num1);
        status1 = fscanf(fp1, "%d", &num1);
    }

    // Copy remaining elements from file2
    while (status2 != EOF) {
        fprintf(fp3, "%d ", num2);
        status2 = fscanf(fp2, "%d", &num2);
    }

    // Close files
    fclose(fp1);
    fclose(fp2);
    fclose(fp3);

    printf("Merged sorted list saved to '%s'.\n", argv[3]);

    return 0;
}
```